Daily Habit Tracker

Introduction

In today's fast-paced world, maintaining consistency in daily habits is challenging. The *Daily Habit Tracker* web application is designed to help users build and sustain positive routines through effective tracking and monitoring. Developed using the Flask framework, the system stores user data, habits, and activity logs in a structured JSON format. Each user has a personalized workspace to manage habits, track streaks, and monitor progress securely. Users can create accounts, log in, and easily add, edit, or delete habits. The application tracks daily completion, calculates progress, and displays it using visual indicators like progress bars and percentages. A key feature is the history module, which provides a detailed view of past habit activity, helping users analyse patterns and improve consistency. Even deleted habits are retained in historical records for complete tracking. Overall, the application is a simple and effective tool that promotes discipline and helps users achieve long-term goals through consistent daily actions.

Project folder structure

```
habit-tracker/
│
├── app.py                  # Main Flask application (your backend lo
├── habits.json             # Data storage (users, habits, logs)
├── requirements.txt        # Python dependencies
│
├── templates/              # All HTML (Jinja2 templates)
│   ├── base.html
│   ├── login.html
│   ├── signup.html
│   ├── index.html          # Home dashboard
│   ├── add.html
│   ├── edit.html           # (optional if using modal)
│   ├── history.html
│
├── static/                 # Static files (CSS, JS, images)
│   ├── logo.jpeg
│   ├── css/
│   │   └── styles.css      # (optional: move inline CSS here)
│   ├── js/
│   │   └── script.js       # (optional: move JS from base.html)
│
```

app.py → Main logic (routes, functions like streak, login, CRUD)

habits. json → Acts like a database

templates/ → UI pages (Jinja2 rendering)

static/ → Images, CSS, JS

User flow diagram

```
Start
  ↓
Visit Website
  ↓
_________________
| Logged in?    |
_________________
   ↓ Yes                ↓ No
   Home Page          Login / Signup
   ↓                    ↓
View Habits          Enter Details
   ↓                    ↓
Mark Complete      Validate User
   ↓                    ↓
Update Streaks     Redirect to Home
   ↓
View Progress
   ↓
Go to History Page
   ↓
Analyze Past Data
   ↓
Add / Edit / Delete Habits
   ↓
Logout
   ↓
End
```

The user begins by accessing the application and either logs in or creates a new account. After authentication, the user is redirected to the home dashboard where they can view and manage their habits. Users can mark habits as completed, which updates their daily progress and streaks automatically. They can also add new habits, edit existing ones, or delete them. The history page allows users to review past performance and analyse consistency over time. Finally, the user can log out, ending the session.

Use case diagram

```
              +----------------------+
              |        User          |
              +----------------------+
                        |
       ----------------------------------------
       |        |         |        |            |
   Sign Up   Login    Logout   View Home   View History
       |        |                   |            |
       |        |                   |            |
       |        |            -------------------------
       |        |            |     |      |         |
       |        |         Add   Edit   Delete   Mark Habit
       |        |         Habit Habit  Habit    Complete
       |        |                                  |
       |        |                           Update Streak
       |        |
       |   Validate Credentials
       |
  Create Account
```

The system has a single primary actor: the **User**. The user can register (Sign Up), log in, and log out of the system. Once authenticated, the user can access the home dashboard to manage habits. This includes adding new habits, editing or deleting existing ones, and marking habits as completed. When a habit is marked complete, the system automatically updates the streak. Additionally, the user can view their past activity through the history feature to analyse progress over time.

Overall functional flow

```
User Accesses Application
          ↓
Authentication Layer
(Login / Signup / Session Check)
          ↓
Load User Data (JSON)
          ↓
————————— Dashboard (Home) —————————
| View Habits                        |
| Mark Habit Complete (Toggle)       |
| → Update Logs                      |
| → Recalculate Streaks              |
| → Save Data                        |
————————————————————————————————————
          ↓
Habit Management
(Add / Edit / Delete Habit)
          ↓
```

```
Update JSON Data Store
          ↓
————————— History Module —————————
| Fetch Past Dates                 |
| Map Habits with Logs             |
| Display Completion Table         |
| Show Daily Totals                |
————————————————————————————————
          ↓
Session Management
(Logout / Continue Session)
          ↓
End
```

The application starts when the user accesses the system and goes through authentication (login or signup). Once authenticated, user-specific data is loaded from a JSON file. The user interacts with the dashboard to view habits, mark them as complete, and track progress. Each action updates logs and recalculates streaks dynamically. Users can also manage habits by adding, editing, or deleting them, which updates the data store. The history module processes stored logs to display past performance and daily completion summaries. Finally, session management allows the user to continue usage or log out, ending the flow.

Project setup

The Daily Habit Tracker application is developed using the Flask framework in Python. To set up the project, a proper development environment is first prepared by installing Python (version 3.8 or above) along with the pip package manager. A virtual environment is then created to isolate project dependencies and ensure smooth package management without affecting the global system configuration.

After setting up the virtual environment, the required libraries such as Flask and Werkzeug are installed using a requirements file. The project follows a structured directory layout, where the main application logic is contained in the app.py file, templates are stored in the templates folder, and static assets such as images are placed in the static directory. The application data is stored in a JSON file (habits.json), which acts as a lightweight database for managing users, habits, and activity logs.

Once the environment and dependencies are ready, the application is executed using the Flask development server. The server runs locally, allowing users to access the application through a web browser using the default address (http://127.0.0.1:5000). On the first run, if the data file does not exist, it is automatically created by the system.

The setup process ensures that the application is easy to install, run, and maintain, making it suitable for development and demonstration purposes without requiring complex database or server configurations.

Code

app.py – Functional Description

The app.py file is the core backend of the Daily Habit Tracker application. It manages user authentication, habit operations, data storage, tracking, and progress calculation using a JSON-based system.

1. Data Handling (Load & Save JSON)

Functionality: Reads and writes users, habits, and activity logs.

Implementation:

- load_data () → Loads JSON data into memory
- save_data () → Updates JSON file with changes

Input: Valid JSON structure
Error Handling: Auto-creates missing file and handles invalid JSON issues.

2. User Authentication

Functionality: Registers new users and verifies login credentials.

Implementation:

- create_user() → Stores user data with hashed password
- login() → Validates credentials
- Uses session["user_id"] for login tracking

Input: Name, email, DOB, gender, password
Error Handling: Duplicate email, missing fields, incorrect password.

3. Session Management

Implementation: Uses a login_required decorator to check session existence.

Error Handling: Redirects to login if session is missing.

4. Habit Management (CRUD)

Functionality: Add, edit, and delete habits.

Implementation:

- Add → Assigns unique ID and stores habit
- Edit → Updates name and description
- Delete → Soft delete using is_deleted flag

Error Handling: Handles invalid IDs and empty inputs.

5. Habit Tracking (Daily Logs)

Functionality: Marks habits as done or not done per day.

Implementation: Updates daily logs based on habit ID and current date.

Error Handling: Handles missing or invalid habit IDs.

6. Streak Calculation

Functionality: Measures user consistency.

Implementation: Checks previous days' logs until a break is found.

Error Handling: Returns 0 if logs are missing.

7. Dashboard & Progress

Functionality: Shows total habits, completed habits, and percentage progress.

Implementation: Completion percentage = (completed / total) * 100

Error Handling: Handles division by zero when no habits exist.

8. History Tracking

Functionality: Shows daily completion history in tabular form.

Implementation: Generates recent dates and maps logs to habits.

Note: Deleted habits may still appear for historical consistency.

login.html - Login Module

The login form sends a **POST request** to the backend route (/login)

Input fields: email, password

- User Lookup:
  - The system queries the database using the provided email.
  - It checks whether a user record exists
- Password Verification
  - The entered password is compared with the stored password
  - If hashing is used, the system verifies using a hash comparison function
- Session Creation
  - If credentials are valid:
    - A session variable (e.g., session['user_id']) is created
    - This keeps the user logged in across pages
- Redirection
  - On success → Redirect to Home page
  - On failure → Reload login page with error message

signup.html - Signup Module

The signup form sends a **POST request** to /signup

Input fields: name, email, dob (date of birth), gender, password, confirm_password.

1. **Input Validation**
   - Checks if all required fields are filled
   - Validates: Email format, Password confirmation, Date of birth within allowed range
2. **Duplicate Check**
   - Queries database to check if email already exists
   - Prevents duplicate accounts
3. **Password Handling**: Password is stored securely (preferably hashed)
4. **User Creation**: Inserts new user record into the database
5. **Post-registration Flow**

   Redirects to: Login page OR automatically logs in user (depending on implementation)

index.html - Index (Home) Module

1. **User Authentication Check**
    - Ensures that a user is logged in (via session)
    - If not logged in → redirects to login page
2. **Fetching Habit Data**
    - Retrieves all habits associated with the current user from the database
    - Each habit contains: id, name, description, streak, done.
3. **Daily Status Calculation**
    - Checks whether each habit is completed for the current day
    - Counts: total habits and completed habits
4. **Performance Calculation**
    - Computes completion percentage: percent = (completed / total) * 100
5. **Rendering the UI**
    - Passes all computed values to the template: habits, total, completed, percent, today
    - Displays: Habit cards, Status indicators (Done/Pending), Streak counters, Progress bar and circular score
6. **Habit Toggle Functionality**
    - Each habit has a checkbox
    - When toggled:
        - A POST request is sent to /toggle
        - Updates today's completion status in the backend
        - Refreshes the page to reflect changes
7. **Edit/Delete Actions**
    - Edit → Opens modal with pre-filled data
    - Delete → Opens confirmation modal
    - Both actions send requests to backend routes

base.html - Base template

It acts as the foundation of the entire application. All other pages extend this template to maintain consistent layout, styling, and structure

1. **Template Inheritance** Uses Jinja blocks:
    - {% block title %}
    - {% block content %}
    - {% block overlays %}
2. **Dynamic Sidebar Rendering** Checks current_user:
    - If logged in → shows Home, Add Habit, History, Logout
    - If not logged in → shows Login and Signup
3. **User Information Display**: current_user.name, current_user. email
4. **Navigation Highlighting**: Uses page variable to highlight active menu item
5. **Styling System**: Layout (grid system), Cards and panels, Forms and buttons.
6. **Overlay Blocks:** {% block overlays %}

history.html - History Module

1. **Fetch User Habits**: Retrieves all habits linked to the logged-in user from the database
2. **Collect Date Range**
    - Generates a list of dates (from signup or recent days), this becomes the table columns
3. **Build Habit-wise Data**
    - Fetches daily completion records
    - Maps each date to: True (completed), False (missed)
4. **Prepare Row Structure**: Name, Description, Streak, Status list, Deleted flag
5. **Calculate Daily Totals**
    - For each date: Counts how many habits were completed
    - Stores as (date, count) pairs
6. **Render Template**:
    - Table with ✔ / — indicators
    - Daily summary cards

edit.html - Edit Habit Module

1. **Receive Request**
    - Edit button (modal or separate page)
    - Sends a POST request with: habit_id, Updated name, Updated description
2. **Fetch Habit**
    - Retrieves the habit using habit_id
    - Ensures it belongs to the current user
3. **Update Data**
    - Old name → New name
    - Old description → New description
4. **Save Changes**: Updates the record in the database
5. **Redirect**: Previous page OR home page
6. **Modal Handling (Frontend)**
    - Pre-fills form using data-* attributes
    - Dynamically sets form action URL

add.html - Add Habit Module

1. **Receive Form Input**
    - POST request from Add Habit form
    - Inputs: name, description
2. **Validate Input**: Habit name is not empty
3. **Create Habit Record**
    - Inserts new habit into database with: User ID, Name, Description, Initial streak = 0
4. **Fetch Updated Habit List**: Retrieves all habits again (to display updated list)
5. **Render Page**: Newly added habit, Existing habits list

habits. json

The **habits.json file** acts as a lightweight data storage system for the application. Instead of using a full database, it stores all user and habit-related information in a structured JSON format.

The file typically contains:

- User data: id, name, email, password, dob, gender
- Habit data: id, name, description, streak, is_deleted
- Daily tracking data
    - Date-wise completion status for each habit
    - Example: "2026-04-21": true
- Reading data whenever a request is made (login, home, history, etc.):
    - The file is opened
    - JSON data is loaded into memory (Python dictionary)
- This allows the application to:
    - Fetch users
    - Retrieve habits
    - Check completion status
- After any update (add, edit, toggle, delete):
    - Adding a habit → new object appended
    - Editing → fields updated
    - Toggle → date status updated
    - Delete → marked or removed
- **Login**: Checks email & password from habits.json
- **Signup**: Adds new user entry
- **Home:** Reads habits, Checks today’s completion
- **Add Habit**: Inserts new habit object
- **Edit Habit**: Updates existing habit fields
- **History**: Reads date-wise tracking, Builds table view

Conclusion

The **Daily Habit Tracker** project is a web-based application designed to help users build consistency by tracking their daily habits in a structured and personalized manner. The system provides individual user accounts, enabling each user to maintain their own set of habits, monitor progress, and analyse performance over time.

The application successfully implements core functionalities such as **user authentication (signup and login), habit creation, editing, deletion, daily tracking, and history visualization**. The dashboard (home page) serves as the central interface where users can mark habits as completed, view their current streaks, and observe their daily performance through visual indicators like progress bars and percentage scores.

A key strength of the system lies in its **user-friendly interface and dynamic interaction**, achieved through template rendering and responsive design. The use of modals for editing and deleting habits enhances usability without interrupting workflow. Additionally, the history module provides meaningful insights by displaying past performance in a tabular format, allowing users to evaluate their consistency over time.

From a technical perspective, the project demonstrates the effective use of:

- **Flask framework** for backend routing and request handling
- **Jinja templating** for dynamic content rendering
- **JSON-based storage (habits.json)** for managing user and habit data
- **Frontend technologies (HTML, CSS, JavaScript)** for interactive UI

Although the system uses a simple JSON file for data storage, which is suitable for small-scale applications, it highlights important concepts such as data organization, user-specific filtering, and CRUD operations. However, for scalability and improved security, the system can be enhanced by integrating a relational database and implementing password hashing.

In conclusion, the project successfully fulfils its objective of creating a **functional, interactive, and personalized habit tracking system**. It demonstrates a strong understanding of full-stack development concepts and provides a solid foundation for future improvements such as analytics, notifications, and mobile integration.